

GENIFY

LECTURE 02

17-July-2025

TOPICS COVERED:

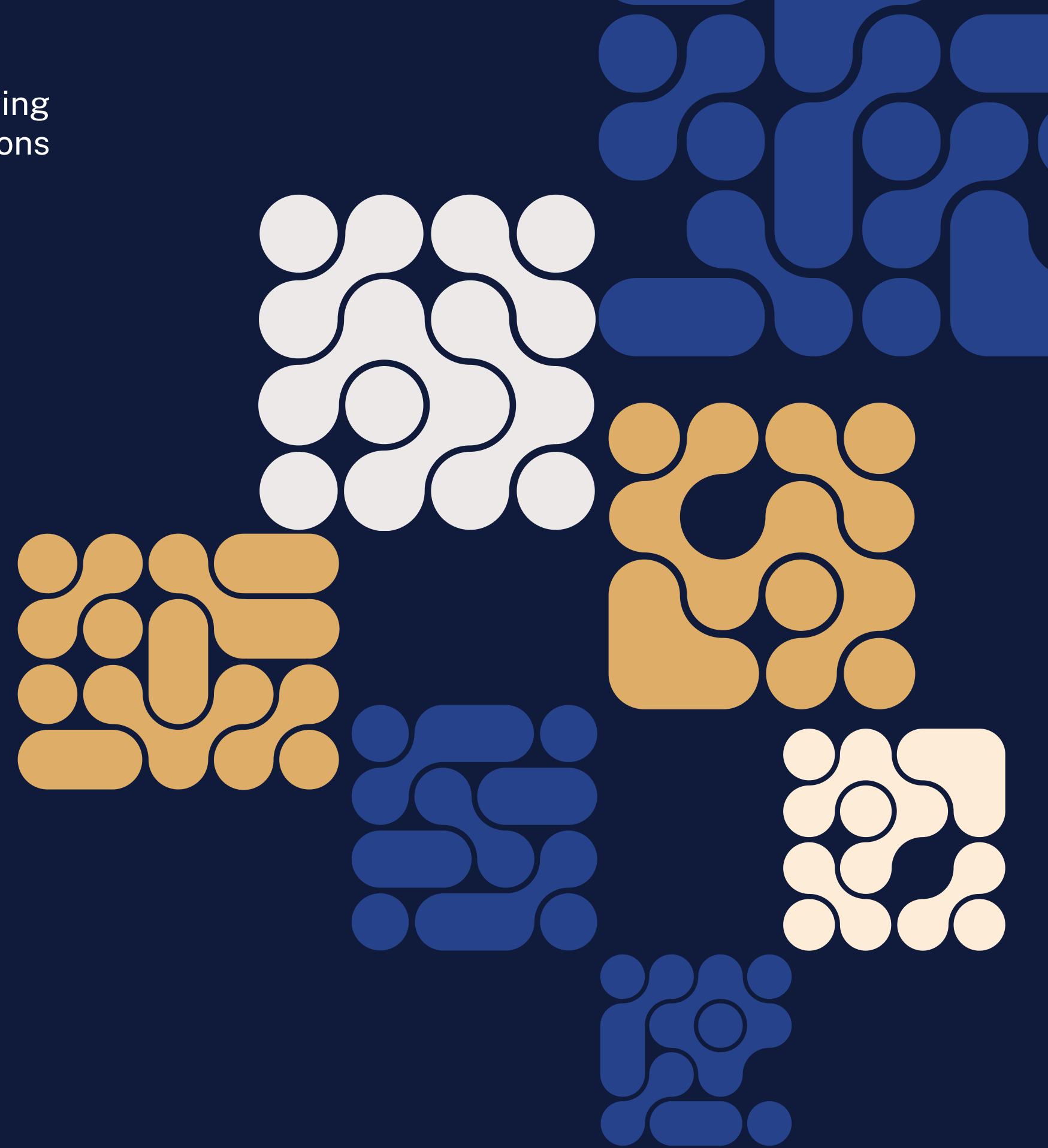
- OBJECT ORIENTED PROGRAMMING (OOP) IN PYTHON
- RISC-V INSTRUCTION SET ARCHITECTURE (ISA)

Presenter

Shahzaib Kashif

Co-Presenter

Shayan Hassan Baig





OBJECT ORIENTED PROGRAMMING

What is Object Oriented Programming?

- Programming paradigm based on objects and classes.
- Organizes code into reusable, modular structures.
- Key concepts: Classes, Objects, Inheritance, Polymorphism, Encapsulation, Abstraction.
- Python supports OOP with simple syntax

ENCAPSULATION



ABSTRACTION



INHERITANCE



POLYMORPHISM



Pillars of OOP

Class and Objects

- Class: Blueprint for objects
- Objects: Instance of a class

```
class Dog :  
    def __init__ ( self, name, age ) :  
        self.name = name  
        self.age = age  
  
    def bark(self) :  
        return f "{self.name} says Woof!"  
  
dog1 = Dog("Buddy",3)  
print(dog1.bark) # Output : Buddy says Woof !
```

Encapsulation

- Restricting access to an object's data
- Use private attributes with `_` or `__` prefix
- Provide public methods (getters/setters) for access

```
class BankAccount :  
    def __init__(self, owner, balance):  
        self.owner      = owner  
        self.__balance  = balance # Private attribute  
  
    def deposit(self, amount):  
        if amount > 0:  
            self.__balance += amount  
        return self.__balance  
  
    def get_balance(self):  
        return self.__balance
```


Inheritance

- Allows a class to inherit attributes and methods from another class
- Promotes code reuse

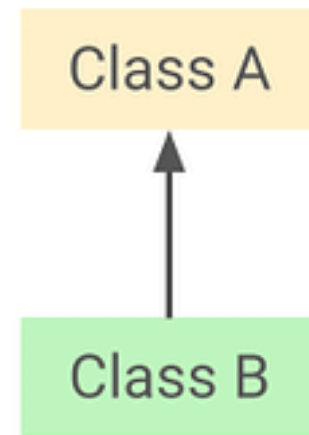
```
class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        pass

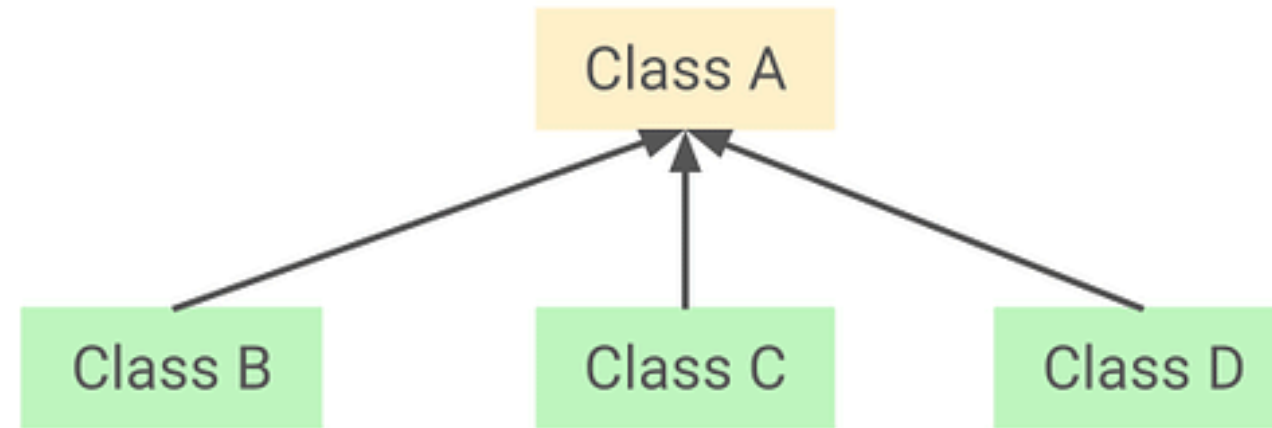
class Cat(Animal):
    def speak(self):
        return f "{self.name} says Meow!"

cat = Cat("Whiskers")
```

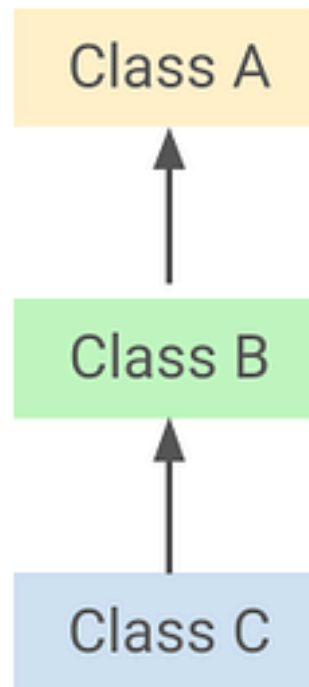
Inheritance - Types



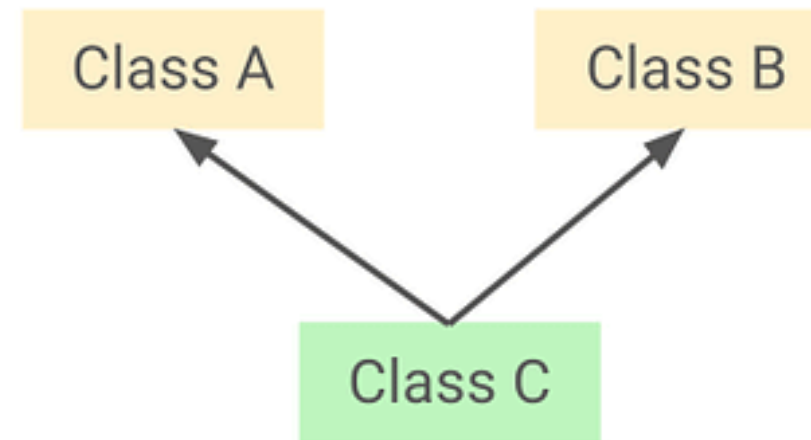
Single Inheritance



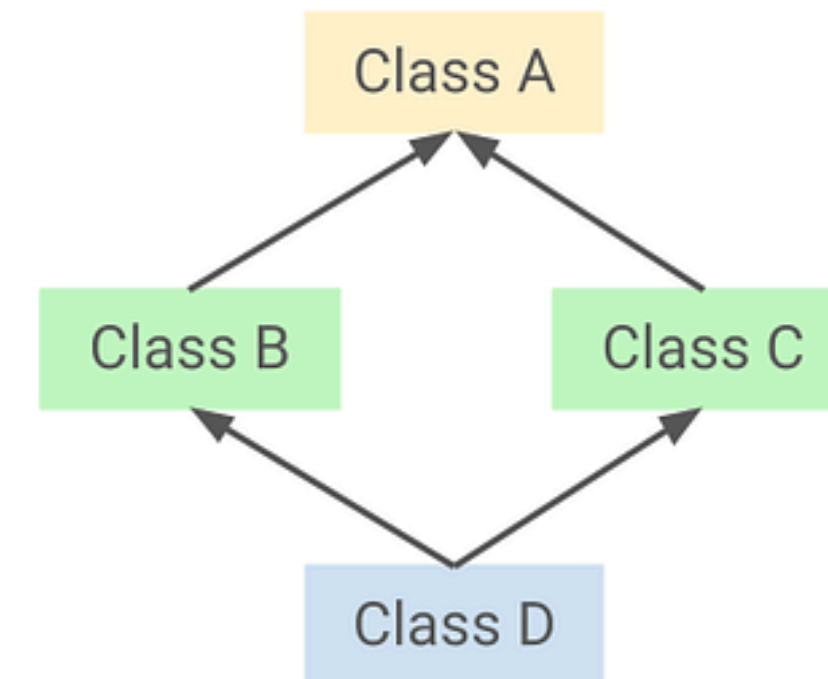
Hierarchical inheritance



Multilevel Inheritance



Multiple Inheritance



Hybrid Inheritance

Polymorphism

- Allows different classes to be treated as instances of the same class
- Achieved through method overriding or method overloading

```
class Dog(Animal):  
    def speak(self):  
        return f "{self.name} says Woof !"
```

```
animals = [Cat("Whiskers") ,Dog("Buddy")]  
for animal in animals :  
    print(animal.speak())
```

==== Output ====

Whiskers says Meow!

Buddy says Woof!

Abstraction

- Hides complex implementation details, exposing only necessary features
- Use abstract base classes (ABC) in Python

```
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self):
        pass

class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius

    def area(self):
        return 3.14 * self.radius ** 2

circle = Circle(5)
print(circle.area()) # Output: 78.5
```

Abstraction

- Hides complex implementation details, exposing only necessary features
- Use abstract base classes (ABC) in Python

```
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self):
        pass

class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius

    def area(self):
        return 3.14 * self.radius ** 2

circle = Circle(5)
print(circle.area()) # Output: 78.5
```

QUIZ

TIME

[HTTPS://TINYURL.COM/GENIFY-OOP](https://tinyurl.com/genify-oop)



RISC-V INSTRUCTION SET ARCHITECTURE

Introduction

How does processors work?

Just like us!

Step01: Fetch

Step02: Decode

Step03: Execute

Step04: Writeback

Why RISC-V?

- Open Source
- Freedom
- Evergrowing ecosystem
- Heavy Industry Adaptation

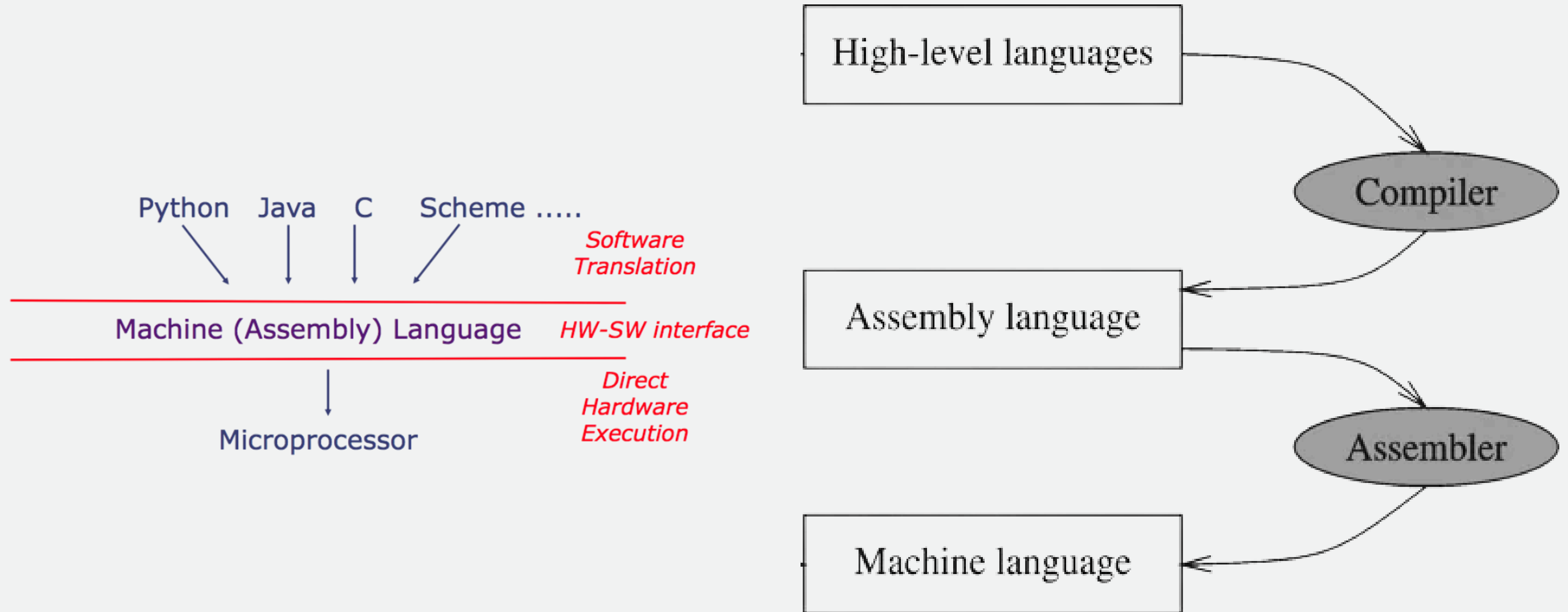
Whats an ISA?

Instruction Set Architectures (ISA) are basically the patterns of bits in simpler terms in which processor understands what to do.

For Example:

- 32/64 bit
- Intel x86
- Arm
- RISC-V

Language Translation



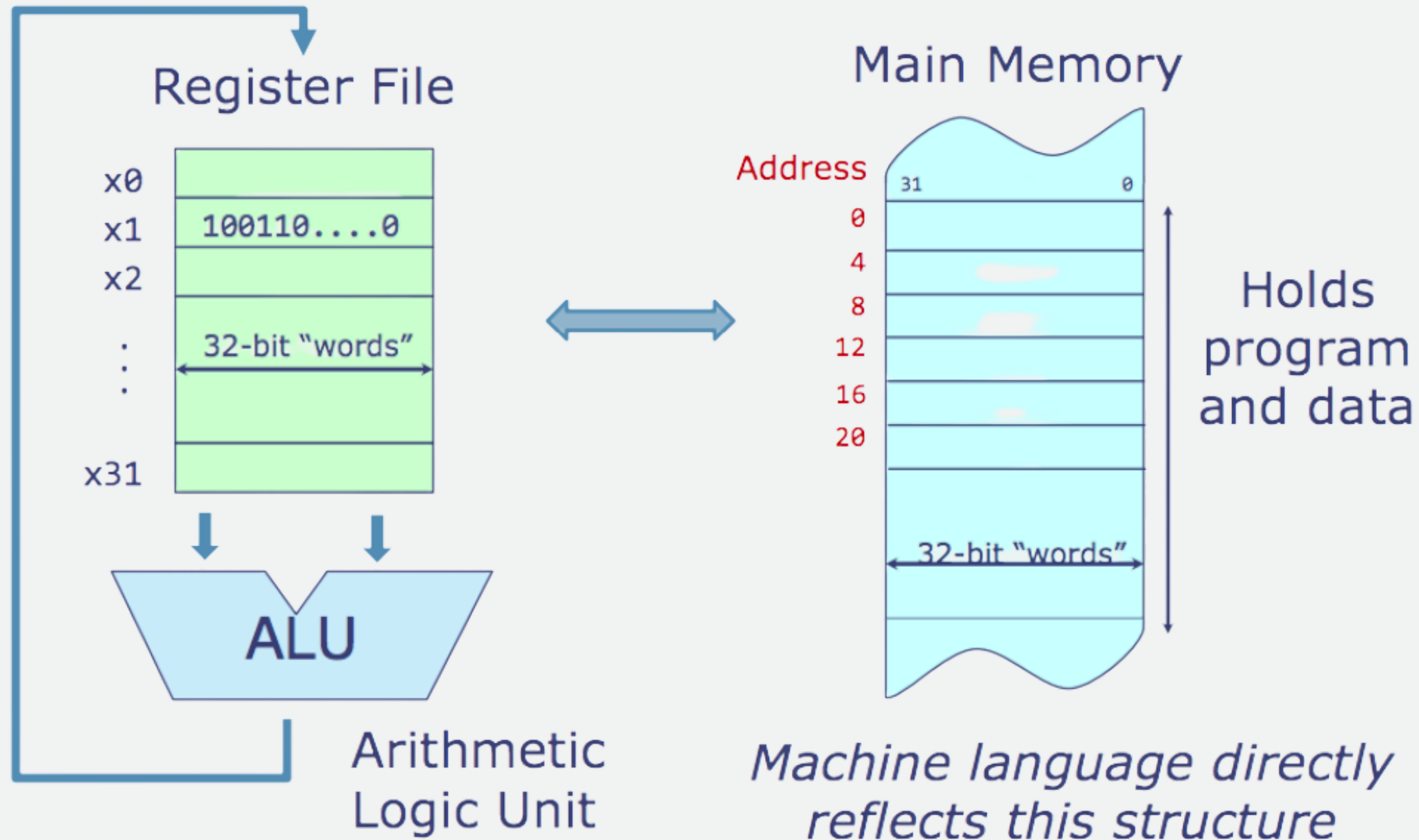
High level language vs Assembly language

- High level language helps programmers bring innovation to the world.
- Just code the high level logic and let the backend handle all the stuff.
- Assembly language is the interface between the software and the hardware.
- Assembly language is more closer towards the hardware.
- Assembly language is more closer towards the hardware.
- Eventually your high level language compiles into an assembly language.

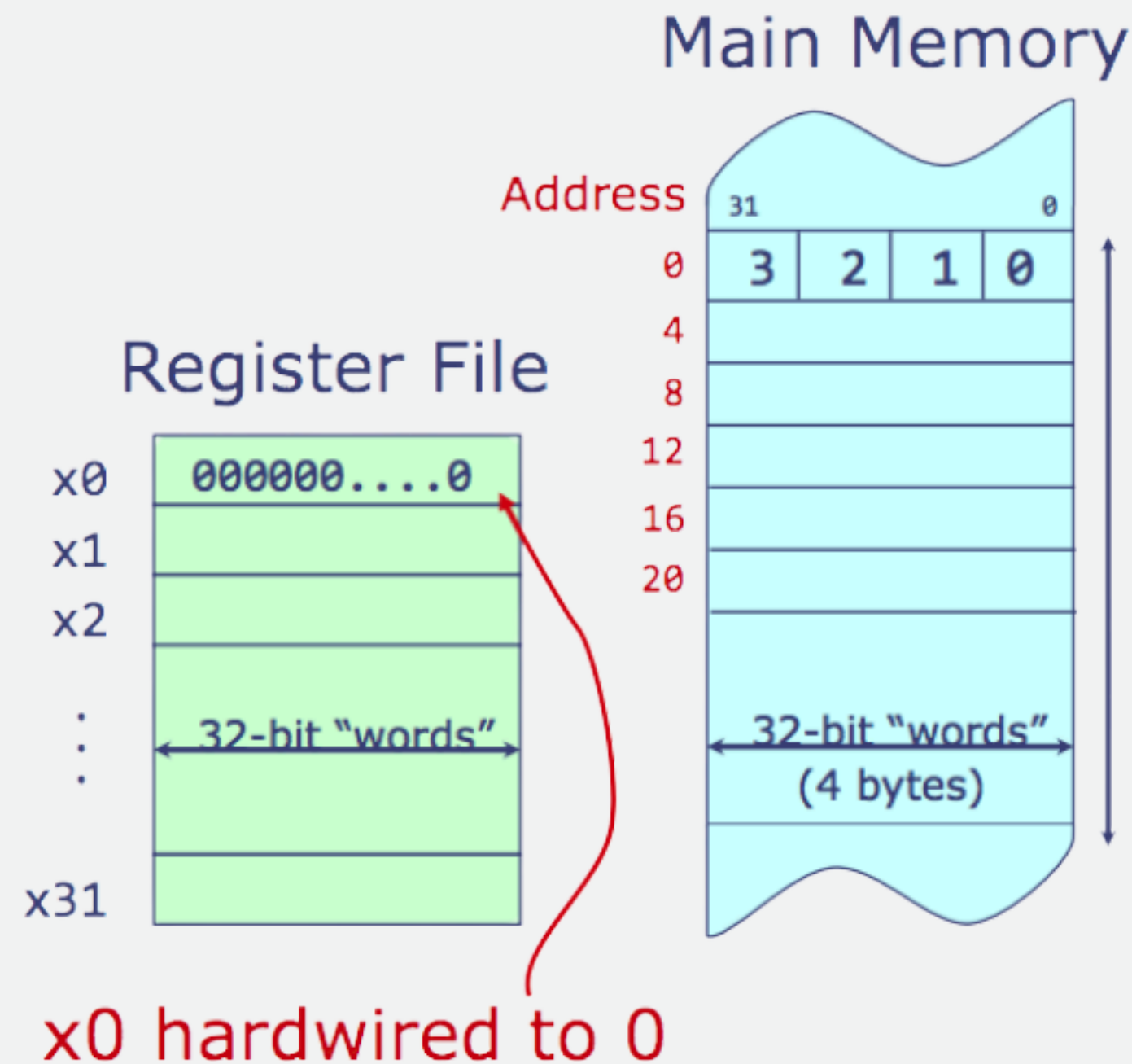
What does processor run?

- Processors generally run bits in 0 or 1.
- Assembly language is further compiled into Machine Code (0s and 1s).
- Again the RISC-V ISA governs how the assembly language is converted into machine language that the processor can understand.

Components of a Microprocessor



RISC-V Processor Storage



Registers:

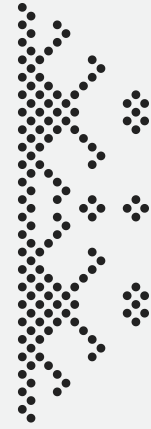
- 32 General Purpose Registers
- Each register is 32 bits wide
- $x0 = 0$

Memory:

- Each memory location is 32 bits wide (1 word)
 - Instructions and data
- Memory is byte (8 bits) addressable
- Address of adjacent words are 4 apart.
- Address is 32 bits
- Can address 2^{32} bytes or 2^{30} words.

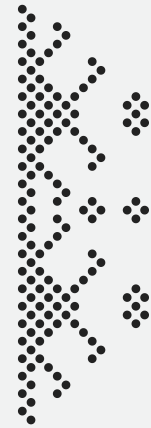
RISC-V ISA: Instructions

Three types of operations



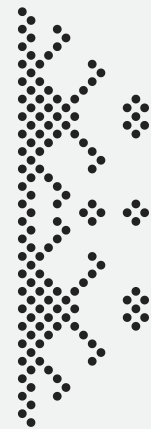
Computational

- Performs arithmetic and logical operations on registers



Loads and Stores

- Move data between registers and Main Memory



Control Flow

- Change the execution order of instructions to support conditional statements and loops

Computational Instructions

- Arithmetic, comparison, logical, and shift operations.
 - Register-Register Instructions:
 - 2 source operand registers
 - 1 destination register

Arithmetic	Comparisons	Logical	Shifts
add, sub	slt, sltu	and, or, xor	sll, srl, sra

- Format: oper dest, src1, src2
 - add x3, x1, x2 ▪ $x3 \leftarrow x1 + x2$
 - slt x3, x1, x2 ▪ If $x1 < x2$ then $x3 = 1$ else $x3 = 0$
 - and x3, x1, x2 ▪ $x3 \leftarrow x1 \& x2$
 - sll x3, x1, x2 ▪ $x3 \leftarrow x1 \ll x2$

Register-Immediate Instructions

- One operand comes from a register and the other is a small constant that is encoded into the instruction.
 - Format: oper dest, src1, const
 - `addi x3, x1, 3` $x3 \leftarrow x1 + 3$
 - `andi x3, x1, 3` $x3 \leftarrow x1 \& 3$
 - `slli x3, x1, 3` $x3 \leftarrow x1 \ll 3$

Format	Arithmetic	Comparisons	Logical	Shifts
Register-Register	add, sub	slt, sltu	and, or, xor	sll, srl, sra
Register-Immediate	addi	slti, sltiu	andi, ori, xori	slli, srli, srai

- No subi, instead use negative constant.
 - `addi x3, x1, -3` ▪ $x3 \leftarrow x1 - 3$

Control Flow Instructions

- Execute `if (a < b):` `c = a + 1`
 `else:` `c = b + 2`
- Need Conditional branch instructions:
 - Format: `comp src1, src2, label`
 - First performs comparison to determine if branch is taken or not: `src1 comp src2`
 - If comparison returns True, then branch is taken, else continue executing program in order.

Instruction	beq	bne	blt	bge	bltu	bgeu
<i>comp</i>	==	!=	<	≥	<	≥

```
        bge x1, x2, else
        addi x3, x1, 1
        beq x0, x0, end
else:   addi x3, x2, 2
end:
```

Assume
x1=a; x2=b; x3=c;

Unconditional Control Instructions

Jumps

- `jal`: Unconditional jump and link
 - Example: `jal x3, label`
 - Jump target specified as label
 - label is encoded as an offset from current instruction
 - Link (To be discussed next lecture): is stored in x3
- `jalr`: Unconditional jump via register and link
 - Example: `jalr x3, 4(x1)`
 - Jump target specified as register value plus constant offset
 - Example: Jump target = $x1 + 4$
 - Can jump to any 32 bit address – supports long jumps

Performing Computations on Values in Memory

$a = b + c$

$x1 \leftarrow \text{load}(\text{Mem}[b])$

$x2 \leftarrow \text{load}(\text{Mem}[c])$

$x3 \leftarrow x1 + x2$

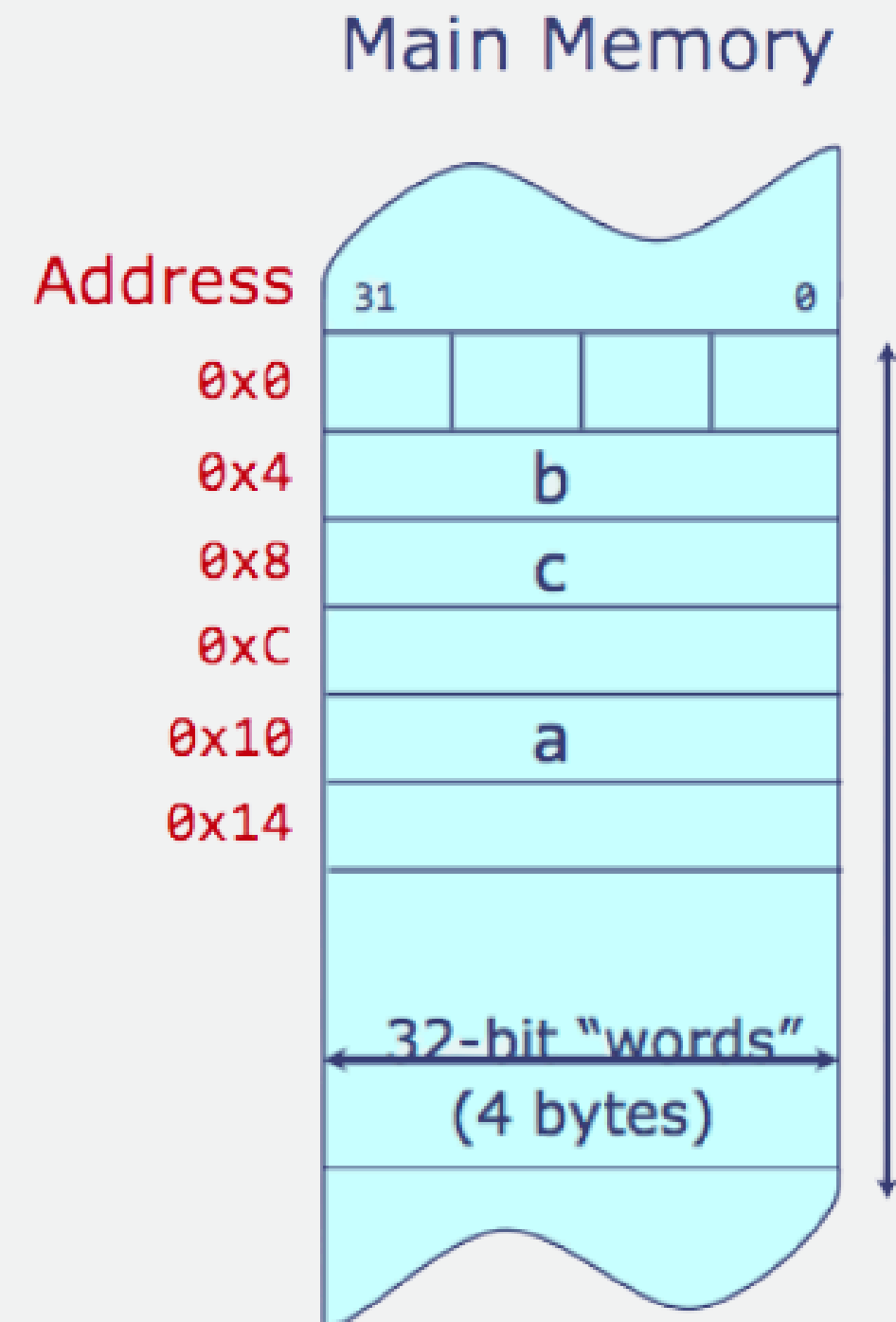
$\text{store}(\text{Mem}[a]) \leftarrow x3$

$x1 \leftarrow \text{load}(0x4)$

$x2 \leftarrow \text{load}(0x8)$

$x3 \leftarrow x1 + x2$

$\text{store}(0x10) \leftarrow x3$



RISC-V Load and Store Instructions

- Address is specified as a <base address, offset> pair;
 - base address is always stored in a register
 - the offset is specified as a small constant
 - Format: lw dest, offset(base) sw src, offset(base)

- Assembly:

```
lw x1, 0x4(x0)
lw x2, 0x8(x0)
add x3, x1, x2
sw x3, 0x10(x0)
```

- Behavior:

```
x1 ← load(Mem[x0 + 0x4])
x2 ← load(Mem[x0 + 0x8])
x3 ← x1 + x2
store(Mem[x0 + 0x10]) ← x3
```

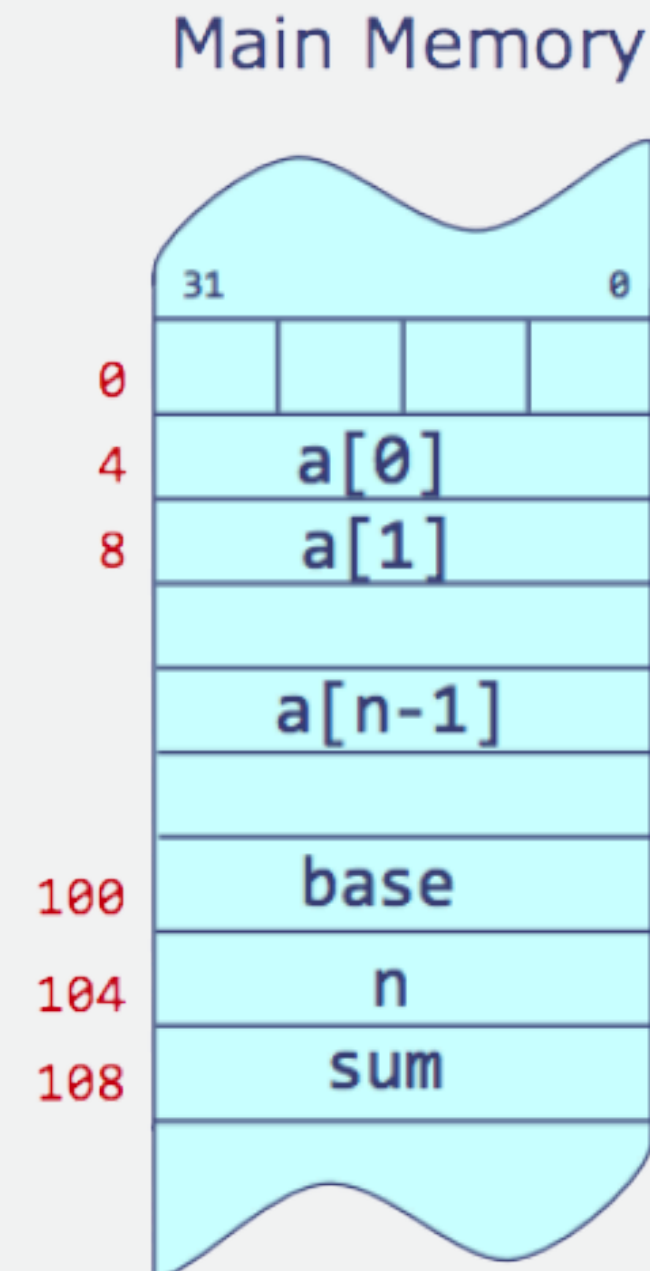
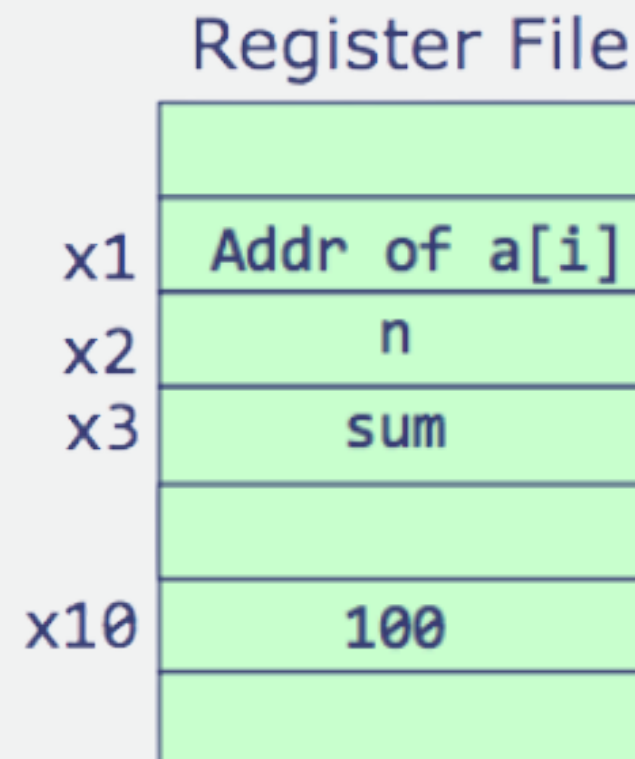
Program to sum Array elements

$\text{sum} = a[0] + a[1] + a[2] + \dots + a[n-1]$

(Assume 100 (address of base) already loaded into x10)

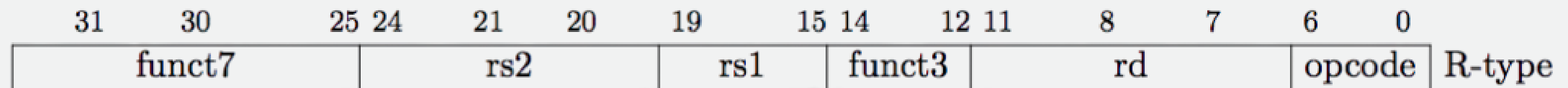
```
lw x1, 0x0(x10)
lw x2, 0x4(x10)
add x3, x0, x0
loop:
    lw x4, 0x0(x1)
    add x3, x3, x4
    addi x1, x1, 4
    addi x2, x2, -1
    bnez x2, loop

sw x3, 0x8(x10)
```



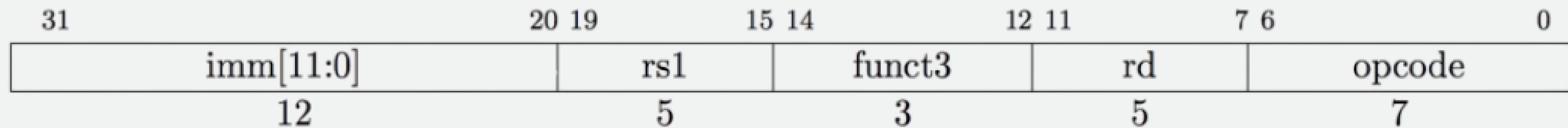
How do computational or R-type instructions convert into machine code?

- 1.add rd, rs1, rs2
- 2.sub rd, rs1, rs2
- 3.and rd, rs1, rs2
- 4.or rd, rs1, rs2
- etc



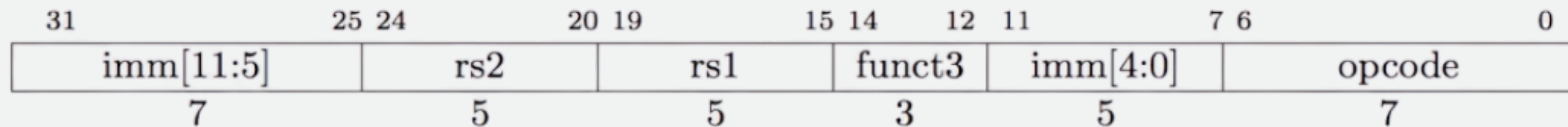
How do Register immediate or I-Type instructions convert into machine code?

- 1.addi rd, rs1, imm
- 2.andi rd, rs1, imm
- 3.lw rd, imm(rs1)
- Etc..



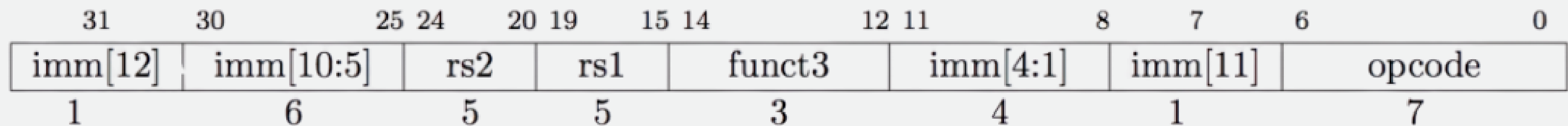
How do store type or S-Type instructions convert into machine code?

1. sw rs2, imm(rs1)



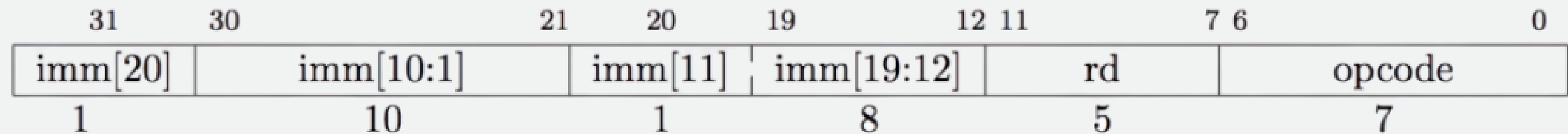
How do branch type or SB-Type instructions convert into machine code?

- 1. beq rs1, rs2, imm
- 2. bne rs1, rs2, imm
- 3. blt rs1, rs2, imm
- Etc..



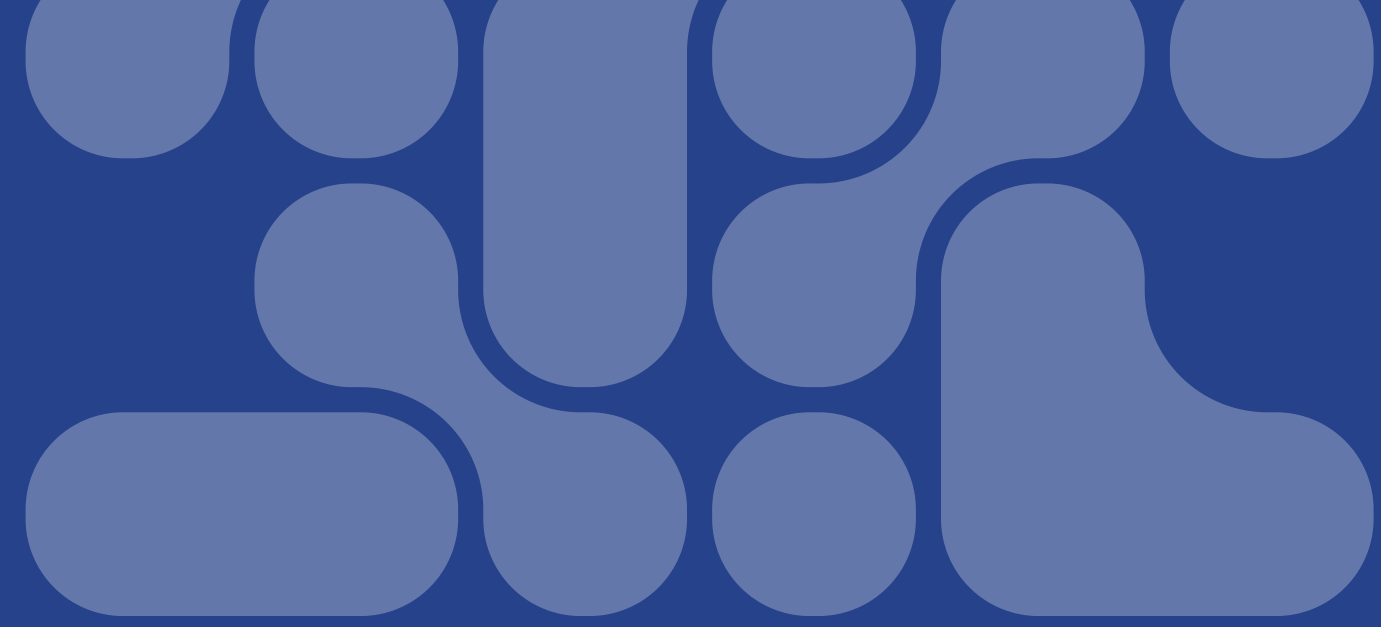
How do unconditional jumps or UJ-Type instructions convert into machine code?

1.jal rd, imm



Complete picture

31	30	25	24	21	20	19	15	14	12	11	8	7	6	0			
funct7				rs2				rs1		funct3		rd			opcode	R-type	
imm[11:0]						rs1		funct3		rd			opcode		I-type		
imm[11:5]				rs2			rs1		funct3		imm[4:0]			opcode		S-type	
imm[12]		imm[10:5]			rs2			rs1		funct3		imm[4:1]		imm[11]		opcode	SB-type
imm[31:12]										rd			opcode		U-type		
imm[20]		imm[10:1]				imm[11]		imm[19:12]				rd			opcode		UJ-type



**Thank you for
your attention!**

Open floor for discussion.

